

Docker Tutorial Channel

Complete Step-by-Step Preparation Roadmap for YouTube

From idea to first upload and beyond: strategy, curriculum, setup, production, publishing and growth

How to use this roadmap

This guide takes you through **10 phases** in a sensible order. Do the phases in sequence during the first 8 weeks, and record at least 5 videos **before** you publish the first one. Having a buffer is the single best protection against burnout and inconsistent uploads.

Phase	What you do	Suggested time
1	Define niche, audience and channel identity	Week 1
2	Research the market and competitors	Week 1
3	Master the content (your own Docker skills)	Weeks 1-3 (ongoing)
4	Design the curriculum and video list	Week 2
5	Set up equipment, software and lab environment	Week 2-3
6	Pre-production: scripts, demos, GitHub repo	Weeks 3-6
7	Production: recording best practices	Weeks 4-7
8	Post-production: editing, thumbnails, captions	Weeks 4-7
9	Publishing, SEO and channel setup	Week 7-8
10	Launch, consistency, analytics and growth	Week 8 onward

Golden rule: A tutorial channel wins by being clear, correct and up to date, not by being flashy. Viewers search for a specific problem ("docker compose not connecting to database"). Your job is to solve it faster and better than the next video.

Phase 1: Define your niche and identity

Docker itself is a broad topic with many existing creators. You stand out by choosing a specific angle and audience.

Step 1.1: Pick your audience

- **Absolute beginners** - students, career switchers, people who have never used a terminal.
- **Developers** - people who write code and need to containerize their apps.
- **DevOps / cloud aspirants** - people heading toward Kubernetes, CI/CD and cloud roles.
- **Language-specific** - Docker for Python, Java Spring Boot, Node.js, or .NET developers.

Step 1.2: Write your channel promise

Complete this sentence: *"I help [audience] learn [outcome] through [style] so they can [result]."*

Example: "I help beginner developers learn Docker through hands-on, project-based videos so they can containerize and deploy their own apps within 30 days."

Step 1.3: Choose your differentiator

- Project-based (build a real app in every module).
- Interview and certification focused (Docker Certified Associate style questions).
- Regional language or bilingual teaching (for example Hindi, Kannada, Tamil, Telugu, English mix) - a large underserved space.
- Troubleshooting-first ("why did my container exit?") style.

- Short, visual, whiteboard-style explanations combined with live demos.

Step 1.4: Channel basics

- [] Channel name: short, memorable, easy to spell, no trademark conflicts (do not use "Docker" as the brand name).
- [] Check the handle is available on YouTube, GitHub, X/Twitter, LinkedIn and Instagram.
- [] Write a 2-line channel description with your promise and upload schedule.
- [] Create a simple logo, banner (2560 x 1440 px) and consistent color palette.

Phase 2: Market and competitor research

Spend 3 to 4 hours here. It will save you weeks later.

Step 2.1: Study existing Docker channels

Search YouTube for: "docker tutorial", "docker for beginners", "docker compose tutorial", "dockerfile explained". Note the top 10 channels and create a simple table:

Question to answer	Why it matters
What is their video length and structure?	Sets viewer expectations for the topic
Which videos have the most views vs. subscribers?	Shows what topics people truly want
What are the top comments asking for?	Free ideas for your next videos
What is outdated or wrong?	Your opportunity: make the updated, correct version
Where do viewers complain about pace or clarity?	Your differentiation

Step 2.2: Keyword research

- Use YouTube search autocomplete, Google "People also ask", and Google Trends.
- Free/low-cost tools: YouTube Studio search insights, vidIQ, TubeBuddy, Ahrefs free keyword tools.
- Collect at least 50 long-tail keywords such as "docker volume vs bind mount" or "docker networking explained".
- Group them into themes; each theme becomes a module in your curriculum.

Step 2.3: Community listening

- Read Reddit (r/docker, r/devops, r/learnprogramming), Stack Overflow top Docker questions, and Docker forums.
- Note the exact wording of beginner problems. Use those words in titles.

Phase 3: Master the content first

You do not need to be the world's top expert, but you must be one clear step ahead of your audience and be accurate. Build a strong foundation before you record.

Step 3.1: Learning checklist

Level	Topics you must be comfortable teaching
Foundation	Why containers exist; containers vs virtual machines; images vs containers; Docker Engine, Docker Desktop, CLI, daemon, registry; namespaces and cgroups at a high level (what makes a container)
Core CLI	docker run, ps, stop, start, rm, exec, logs, inspect, pull, push, images, rmi, system prune; port mapping; environment variables; container lifecycle
Images	Dockerfile instructions (FROM, RUN, COPY, ADD, WORKDIR, ENV, ARG, EXPOSE, CMD, ENTRYPOINT, USER, HEALTHCHECK); layers and cache; .dockerignore; multi-stage builds; choosing base images (alpine, slim, distroless)
Data	Volumes vs bind mounts vs tmpfs; persisting database data; backups
Networking	Bridge, host, none, custom networks; container DNS by service name; port publishing; connecting multiple containers
Docker Compose	compose.yaml structure; services, networks, volumes; depends_on with healthchecks; env files; profiles; build and override files; Compose Watch for development
Registries	Docker Hub, GitHub Container Registry, private registries; tagging strategy; pushing and pulling; image digests
Security	Non-root users; minimal images; scanning (Docker Scout, Trivy); secrets handling; read-only file systems; avoiding latest tag; supply-chain basics
Advanced	BuildKit and buildx; multi-platform images (amd64/arm64); build cache and secrets mounts; resource limits; logging drivers; debugging failed containers; Docker Init; Docker Model Runner and AI/ML containers (optional)
Ecosystem	CI/CD with GitHub Actions; deploying to a cloud VM; intro to Docker Swarm; bridge to Kubernetes; dev containers

Step 3.2: Learning sources

- **Official documentation** at docs.docker.com - your primary source of truth. Always verify commands against it.
- Docker's official "Get Started" guides and Docker Labs / Play with Docker for practice.
- Book suggestions: *Docker Deep Dive* (Nigel Poulton), *Docker in Practice* (Miell and Sayers).
- Build 3 to 5 real projects yourself before recording (see Phase 4 projects).

Step 3.3: Keep an accuracy log

Keep a notes file for every command you plan to show: the command, the expected output, the Docker version you tested on, and any gotchas. Docker updates often (Compose v2 syntax, the `docker compose` command replacing the old `docker -compose`, the removal of the `top-level version:` key). Mentioning the version you use in each video keeps your content trustworthy.

Teaching test: Before recording any topic, explain it out loud in 2 minutes to an imaginary beginner without notes. Whatever you cannot explain simply, you do not yet understand well enough to teach.

Phase 4: Design the curriculum

Plan a structured playlist-based course. Viewers who see a clear path are far more likely to subscribe. Below is a proven 6-module structure with about 30 videos. Aim for videos of 8 to 20 minutes, except project videos which can be longer.

Module 1: Foundations (5 videos)

- What is Docker and why do we need it? (containers vs VMs)
- Installing Docker on Windows, macOS and Linux
- Your first container: hello-world, nginx, and how docker run works
- Essential Docker CLI commands (hands-on cheat sheet)
- Images, containers, registries: the full mental model

Module 2: Building Images (6 videos)

- Writing your first Dockerfile
- Layers and build cache: make builds 10x faster
- CMD vs ENTRYPOINT, COPY vs ADD, ARG vs ENV
- .dockerignore and keeping images small
- Multi-stage builds explained
- Pushing images to Docker Hub and GitHub Container Registry

Module 3: Data and Networking (5 videos)

- Docker volumes vs bind mounts
- Running a database (Postgres/MySQL) with persistent storage
- Docker networking explained: bridge, host, none
- Container-to-container communication with custom networks
- Environment variables, .env files and basic secrets

Module 4: Docker Compose (5 videos)

- Docker Compose from scratch
- Full-stack app: frontend + backend + database with Compose
- Healthchecks and startup ordering
- Compose profiles, override files and Compose Watch
- Debugging Compose: logs, exec, common errors

Module 5: Production Practices (5 videos)

- Docker security best practices (non-root, minimal images, scanning)
- Optimizing image size: alpine vs slim vs distroless
- CI/CD: build and push images with GitHub Actions
- Deploy a Dockerized app on a cloud VM (AWS EC2 / any VPS)
- Resource limits, logging and monitoring basics

Module 6: Beyond the Basics (4 videos)

- BuildKit, buildx and multi-platform images
- Docker Swarm vs Kubernetes: what to learn next
- Dev containers in VS Code
- Top 25 Docker interview questions and answers

Step 4.2: Capstone projects (recommended)

- **Project A:** Containerize a simple Python Flask or Node.js API.
- **Project B:** React + Node + PostgreSQL full stack app with Compose.
- **Project C:** Static site served by Nginx with multi-stage build.
- **Project D:** CI/CD pipeline that builds, scans and pushes an image on every commit.

Step 4.3: Launch batch and content pillars

Do not launch with a single video. Plan a **launch batch of 3 to 5 videos** published in the first week, chosen from Module 1 and one high-search-volume video such as "Docker Compose tutorial". Then commit to a regular rhythm.

Content pillar	Share	Purpose
Core course videos	60%	Structured learning, playlist watch time
Problem-solving / error fixes	20%	Search traffic from people stuck on errors
Projects and real-world demos	15%	Portfolio value, high retention
Shorts / quick tips	5%	Discovery by new viewers

Phase 5: Equipment, software and lab setup

Start with the minimum. Audio quality matters far more than video quality for tutorials. Upgrade only when the channel earns it.

Step 5.1: Hardware

Item	Starter option	Notes
Computer	Laptop or desktop with 16 GB RAM, SSD, quad-core CPU	Docker plus screen recording is heavy; 8 GB is painful
Microphone	USB mic (for example Fifine K669, Samson Q2U, Audio-Technica ATR2100x)	Most important purchase; or a decent wired earphone mic to begin
Webcam (optional)	1080p webcam or phone as a webcam	A small face cam builds trust but is not mandatory
Lighting	One softbox or ring light in front of you	Only needed if you show your face
Second monitor	Optional	Helps you read the script while recording
Pop filter / quiet room	Foam pop filter; record in a small carpeted room	Reduces echo and plosives

Step 5.2: Software

Purpose	Free options	Paid options
Screen recording	OBS Studio	Camtasia, ScreenFlow
Editing	DaVinci Resolve, CapCut, Shotcut	Camtasia, Final Cut Pro, Premiere Pro
Audio cleanup	Audacity, DaVinci Fairlight, Adobe Podcast Enhance	Adobe Audition
Thumbnails	Canva, Figma, Photopea, GIMP	Photoshop
Diagrams / slides	Excalidraw, draw.io, Google Slides, tldraw	Figma, Keynote
Terminal	Windows Terminal, iTerm2, Warp, VS Code terminal	-
Notes and planning	Notion, Obsidian, Google Docs, Trello	-

Step 5.3: Recording environment for Docker demos

- Use a **clean machine or a clean user profile** so viewers see a predictable environment.
- Install the latest stable Docker Desktop (or Docker Engine on Ubuntu) and verify with `docker version` and `docker compose version`.
- Increase terminal font size (18 to 22 pt) and use a high-contrast theme.
- Use a short prompt (hide long paths and hostnames) so commands are readable.
- Set the screen to 1920 x 1080 and record at 30 fps (60 fps is unnecessary).
- Turn off notifications (Do Not Disturb) and close browser tabs, chat apps and anything private.
- Never show secrets, tokens or real credentials; use dummy values and rotate any that leak.
- Optional: a Linux VM or cloud VPS for the deployment videos (a small instance is fine).

Useful terminal setup for readable recordings:

```
# Shorten the prompt in bash/zsh for cleaner recordings
export PS1='\$ '

# Verify your environment before every session
docker version
docker compose version
docker system df
```

Step 5.4: Create your GitHub repository

- Create one public repo (for example `docker-tutorial-course`) with one folder per video: `01-intro/`, `02-install/`, and so on.
- Put working code, Dockerfiles, compose files, and a README with commands and the Docker version used.
- Link the exact folder in every video description. This builds trust and lets viewers follow along.
- Add a LICENSE (MIT is common) and a CONTRIBUTING note if you want community fixes.

Phase 6: Pre-production (scripts and demos)

Good tutorials are planned. Unplanned tutorials ramble, contain mistakes and lose viewers within 30 seconds.

Step 6.1: The standard video structure

Segment	Duration	What to do
Hook	0:00-0:30	State the problem and the promise: "By the end you will run a full app with one command."
Context	0:30-1:30	Brief why-it-matters and prerequisites; show the final result first
Concept	1-4 min	Explain with a simple diagram or analogy (shipping container, lunchbox)
Demo	Majority	Live step-by-step on screen; type the commands, do not paste silently
Recap	Last minute	Summarize 3 to 5 key points and show the cheat sheet
Call to action	Final 15 sec	Point to the next video in the series; ask them to comment with questions

Step 6.2: Script workflow

- Write an outline first, then bullet-point notes for the demo; full word-for-word scripts are best for hooks, intros and concept explanations.
- Keep sentences short and speak the way you talk to a friend.
- Use analogies: an image is a recipe or class; a container is the cooked dish or object.
- Mark on-screen moments: [DIAGRAM], [TERMINAL], [ZOOM], [CALLOUT].
- Rehearse the demo end-to-end at least twice **before** recording. Time it.

Step 6.3: Prepare each demo

- [] Run all commands on a clean environment and confirm they work.
- [] Prepare the starting state (repo, files, images pulled if slow).
- [] Pre-pull large images to avoid dead time, but tell viewers the download will take time on their side.
- [] Prepare intentional errors for troubleshooting segments (error teaches more than success).
- [] Create diagrams in advance: container vs VM, image layers, network topology, Compose architecture.
- [] Write a one-page cheat sheet or command list to offer as a free download in the description.

Step 6.4: Video template file

```
TITLE:                (primary keyword first)
AUDIENCE/LEVEL:       Beginner | Intermediate
DOCKER VERSION:       e.g. Docker Engine 28.x / Compose v2
PROBLEM SOLVED:       ...
HOOK (30 sec):        ...
PREREQUISITES:        ...
OUTLINE:              1) ... 2) ... 3) ...
DEMO COMMANDS:        (list in order)
GOTCHAS:              ...
RECAP POINTS:         ...
NEXT VIDEO:           ...
GITHUB FOLDER:        ...
```

Phase 7: Production (recording)

Step 7.1: Recording best practices

- **Record audio first-class:** mic 10 to 15 cm from your mouth, gain set so peaks reach about -12 dB, and do a 10-second silence recording to capture room noise.
- Record in **short segments** (one concept per take) rather than one long run; it is easier to re-record and edit.
- Zoom the terminal and editor (Ctrl/Cmd + plus) so text is readable on a phone screen.
- Type commands at a normal pace and narrate what you are typing and why before pressing Enter.
- When a command is slow (image pull, build), stop recording or plan to cut/speed it up in editing.
- If you make a mistake, pause 3 seconds, then repeat the sentence. The pause makes it easy to find and cut.
- Show **expected output** and explain how to read it (for example, what each column of docker ps means).
- Use a mouse highlight or cursor zoom so viewers can follow where you click.
- Face cam is optional; if used, keep it small in a corner and look at the lens during intros and outros.

Step 7.2: Record in batches

Choose one or two days per week for recording only. Recording 3 to 4 videos in one session is more efficient than one per day because your setup, voice and energy stay consistent. Keep editing and publishing on separate days.

Step 7.3: OBS Studio quick settings

Setting	Recommended value
Base and output resolution	1920 x 1080
Frame rate	30 fps
Video encoder	Hardware (NVENC/Apple VT/AMF) if available; otherwise x264
Recording format	MKV (safe against crashes), then remux to MP4
Bitrate	8,000 to 12,000 Kbps for screen content
Audio	48 kHz, mic and desktop audio on separate tracks
Filters on mic	Noise suppression (RNNoise), noise gate, compressor, limiter

Phase 8: Post-production

Step 8.1: Editing workflow

- **Rough cut:** remove mistakes, long silences, and slow waits.
- **Pacing:** speed up boring parts (downloads, builds) 4x with an on-screen note such as "sped up".
- **Visual aids:** zoom-ins on commands, highlight boxes on important output, arrows and callouts.
- **Chapters:** add timestamps so viewers can jump to the section they need (huge for tutorials).
- **Audio:** normalize to about -14 LUFS, reduce noise, add very subtle background music only during intro/outro.
- **Captions:** auto-generate then correct them; captions help accessibility, non-native English viewers and SEO.
- **Intro:** keep it under 5 seconds or skip it. Start with the hook.

Step 8.2: Thumbnails that get clicks

- Size 1280 x 720 px, under 2 MB, JPG or PNG.
- Use a consistent template so your channel is recognizable.
- Big readable text: 2 to 4 words at most (for example "Docker Compose", "Fix This Error").
- Use the Docker whale and tool logos sparingly and check current brand guidelines; avoid implying official endorsement.
- High contrast, one clear focal point; test how it looks at tiny size on a phone.
- Prepare 2 thumbnail variants and use YouTube's Test and Compare feature when available.

Step 8.3: Quality control checklist before upload

- [] Every command works exactly as shown; final code is pushed to GitHub.
- [] Audio is clear on phone speakers and headphones.
- [] No secrets, IP addresses, tokens or personal data visible on screen.
- [] Text on screen is readable at 360p.
- [] Chapters, captions and end screens are ready.
- [] Description and pinned comment drafted.

Phase 9: Channel setup, publishing and SEO

Step 9.1: Set up your channel

- [] Create a YouTube channel (a Brand Account is recommended so you can add managers later).
- [] Upload profile picture (800 x 800) and banner (2560 x 1440).
- [] Write the About section with keywords: Docker, containers, DevOps, tutorials, plus your language and audience.
- [] Add links: GitHub, LinkedIn, website, community.
- [] Verify your account (phone) to unlock longer videos and custom thumbnails.
- [] Create playlists per module (Module 1: Docker Basics, etc.) and one "Start Here" playlist.
- [] Create a channel trailer (60 to 90 seconds) and a featured section.

Step 9.2: Video SEO formula

Element	Guideline	Example
Title	55-65 chars, keyword first, clear benefit	Docker Compose Tutorial for Beginners (Full Project)
Description first 2 lines	Include keyword and what the viewer will learn; appears in search	In this video you will learn how to run a multi-container app with Docker Compose...
Description body	Timestamps, GitHub link, prerequisites, playlist link, commands used	0:00 Intro / 1:20 Compose file / ...
Tags	Minor factor; add 5-10 relevant ones and common misspellings	docker compose, docker tutorial, dockercompose
Hashtags	2-3 max	#docker #devops #containers
Pinned comment	Repo link plus a question to encourage discussion	What should I cover next?
End screen	Best next video plus subscribe	Next: Docker Volumes explained

Step 9.3: Sample description template

In this video, you will learn [outcome] using Docker.

Code & commands: <https://github.com/YOURNAME/docker-tutorial-course/tree/main/05-compose>

Docker version used: 28.x | Compose v2

Previous video: [link] Next video: [link]

TIMESTAMPS

0:00 Intro

1:10 What is Docker Compose?

3:40 Writing compose.yaml

8:15 Running and testing

12:30 Recap

Prerequisites: Docker installed (see video 02)

#docker #devops #containers

Step 9.4: Upload settings

- Set the audience correctly ("Not made for kids").
- Add the playlist, category (Education or Science and Technology), language, captions and license.
- Schedule uploads for consistent day and time; check your audience's active hours in YouTube Studio later.
- If you use AI-generated voice or realistic altered content, follow YouTube's disclosure requirements.

Phase 10: Launch, consistency and growth

Step 10.1: 30-day launch plan

Days	Actions
1-3	Publish the first 3 to 5 videos (launch batch). Share with friends, colleagues, college groups. Ask for honest feedback.
4-10	Post in relevant communities where allowed (LinkedIn, dev communities, Discord/Slack groups, Reddit within each subreddit's self-promotion rules). Provide value, not spam.
11-20	Publish 2 videos per week. Reply to every comment. Note questions and turn the best into new videos.
21-30	Review analytics, improve titles/thumbnails on underperformers, publish one troubleshooting video and one Short.

Step 10.2: Consistency system

- Choose a sustainable cadence: 1 to 2 videos per week is enough. Consistency beats intensity.
- Keep a 3 to 5 video buffer at all times.
- Batch your work: research day, record day, edit day, publish day.
- Track ideas in a running backlog (Notion or Trello) with keyword, difficulty and priority.

Step 10.3: Analytics to watch

Metric	Where	What it tells you
Impressions click-through rate (CTR)	YouTube Studio > Reach	Are titles and thumbnails compelling? (2 to 10% is a typical range)
Average view duration / retention	Studio > Engagement	Where viewers drop off; fix intros and slow segments
Traffic sources	Studio > Reach	Search vs suggested vs external; shows if your SEO works
Returning viewers	Studio > Audience	Loyalty of your course-style content
Comments and questions	Studio > Comments	Direct source of next video topics

Step 10.4: Growth tactics

- Turn each long video into 1 to 3 Shorts (one command, one concept, one tip).
- Publish a written blog version or GitHub README for each video; it helps search visibility.
- Collaborate with creators in adjacent topics (Linux, Kubernetes, Python, cloud) for cross-promotion.
- Update popular videos: pin a comment with corrections or new versions, or re-record when tooling changes.
- Build an email list or community (Discord/Telegram) with your cheat sheets as a free lead magnet.
- Answer Docker questions on Stack Overflow and Reddit and link your video only when it truly answers the question.

Step 10.5: Monetization roadmap

YouTube Partner Program eligibility has tiers that change over time; check the current requirements in YouTube Studio. Typically there is an earlier tier with fewer subscribers for channel memberships and fan funding, and a higher tier for ad revenue. Do not wait for ads; tutorial channels often earn faster through:

-
- Affiliate links (cloud hosting credits, courses, hardware; always disclose).
 - Sponsorships from dev tool and cloud companies.
 - Your own paid course, cohort or e-book (Docker Interview Guide).
 - Freelance consulting or job opportunities created by your visible expertise.
 - Memberships and community perks (early access, code reviews).

Legal, ethical and quality notes

- **Copyright:** use only music, images and footage you own or that are licensed for commercial use (YouTube Audio Library, Pixabay, Epidemic Sound). Do not copy others' scripts or slides.
- **Trademarks:** the Docker name and whale logo belong to Docker, Inc. Referring to Docker is fine; do not present your channel as official.
- **Accuracy:** pin corrections quickly if you make a mistake. Honesty builds long-term trust.
- **Security:** teach safe defaults. Never recommend running containers as root with the Docker socket mounted, or disabling security features, without warning about risks.
- **Disclosures:** tell viewers about sponsorships and affiliate links.

Master pre-launch checklist

Strategy

- ☐ Audience and promise written
- ☐ Channel name and handles secured
- ☐ 10 competitor channels analysed
- ☐ 50+ keywords collected

Content

- ☐ Curriculum with 25-30 videos planned
- ☐ Launch batch of 3-5 videos selected
- ☐ GitHub repo created
- ☐ Accuracy log started
- ☐ Cheat sheet prepared

Equipment and software

- ☐ Microphone tested
- ☐ OBS configured
- ☐ Docker installed and verified
- ☐ Editing software installed
- ☐ Thumbnail template ready

Production

- ☐ 5 videos recorded and edited before launch
- ☐ Audio, captions and chapters done
- ☐ Descriptions and pinned comments drafted
- ☐ QC checklist passed

Channel

- ☐ Banner, logo, About and links added
- ☐ Playlists created
- ☐ Trailer uploaded
- ☐ Upload schedule announced

Appendix A: 8-week timeline at a glance

Week	Focus	Deliverables
1	Niche, audience, competitor research, keywords	Channel promise, competitor sheet, keyword list, channel name
2	Curriculum design and equipment	30-video outline, mic and OBS tested, repo created
3	Deep learning and demo building	Projects A and B complete, accuracy log for Modules 1-2
4	Script and record Module 1 videos	Videos 1-3 recorded
5	Record and edit	Videos 4-5 recorded; videos 1-3 edited; thumbnails ready
6	Editing and channel setup	Branding, playlists, trailer, SEO templates
7	QC, buffer building	5 videos fully finished; 2 more scripted
8	Launch	Publish batch; community sharing; feedback review

Appendix B: Docker command cheat sheet (share this with viewers)

```
# Containers
docker run -d --name web -p 8080:80 nginx      # run in background, map port
docker ps -a                                  # list containers
docker logs -f web                            # follow logs
docker exec -it web sh                        # open shell inside
docker stop web && docker rm web               # stop and remove

# Images
docker build -t myapp:1.0 .                   # build from Dockerfile
docker images                                 # list images
docker tag myapp:1.0 USER/myapp:1.0          # tag for registry
docker push USER/myapp:1.0                   # push to registry

# Data and networks
docker volume create mydata
docker network create appnet
docker run -v mydata:/var/lib/postgresql/data postgres

# Compose
docker compose up -d --build
docker compose ps
docker compose logs -f
docker compose down -v

# Cleanup
docker system prune -a --volumes               # careful: removes unused data
```

Appendix C: Starter Dockerfile (multi-stage Node.js example)

```
# Build stage
FROM node:22-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# Runtime stage
FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Appendix D: Sample compose.yaml

```
services:
  web:
    build: .
    ports:
      - "3000:3000"
    environment:
      DATABASE_URL: postgres://app:secret@db:5432/appdb
    depends_on:
      db:
        condition: service_healthy
  db:
    image: postgres:16
    environment:
      POSTGRES_USER: app
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: appdb
    volumes:
      - dbdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U app -d appdb"]
      interval: 5s
      retries: 5

volumes:
  dbdata:
```

Appendix E: Useful links

- Docker documentation: docs.docker.com
- Play with Docker (browser lab): labs.play-with-docker.com
- Docker Hub: hub.docker.com
- YouTube Creator Academy and Help Center: support.google.com/youtube
- YouTube Audio Library (free music): inside YouTube Studio
- OBS Studio: obsproject.com
- DaVinci Resolve: blackmagicdesign.com

Verify all tool versions, prices and YouTube program requirements at the time you start, since they change frequently.